6.7.1 Pessimistic and optimistic locking

Isolation levels entail implicit locking, whether it involves physical locks (like 2PL) or data anomaly detection (MVCC). To coordinate state changes, application-level concurrency control makes use of explicit locking, which comes in two flavors: pessimistic and optimistic locking.

6.7.1.1 Pessimistic locking

As previously explained, most database systems already offer the possibility of manually requesting shared or exclusive locks. This concurrency control is said to be *pessimistic* because it assumes that conflicts are bound to happen, and so they must be prevented accordingly.

> As locks can be released in a timely fashion, exclusive locking is appropriate during the last database transaction of a given long conversation. This way, the application can guarantee that, once locks are acquired, no other transaction can interfere with the currently locked resources.
>
> Acquiring locks on critical records can prevent non-repeatable reads, lost updates, as well as read and write skew phenomena.

6.7.1.2 Optimistic locking

Undoubtedly a misnomer (albeit rather widespread), optimistic locking does not incur any locking at all. A much better name would be optimistic concurrency control since it uses a totally different approach to managing conflicts than pessimistic locking.

MVCC is an optimistic concurrency control strategy since it assumes that contention is unlikely to happen, and so it does not rely on locking for controlling access to shared resources. The optimistic concurrency mechanisms detect anomalies and resort to aborting transactions whose invariants no longer hold.

While the database knows exactly which row versions have been issued in a given time interval, the application is left to maintaining a *happens-before* event ordering. Each database row must have an associated version, which is locally incremented by the logical transaction. Every modifying SQL statement (update or delete) uses the previously loaded version as an assumption that the row has not been changed in the meanwhile.

Because even the lowest isolation level can prevent write-write conflicts, only one transaction is allowed to update a row version at any given time. Since the database already offers monotonic updates, the row versions can also be incremented monotonically, and the application can detect when an updating record has become stale.

The optimistic locking concurrency algorithm looks like this:

- When a client reads a particular row, its version comes along with the other fields.
- Upon updating a row, the client filters the current record by the version it has previously loaded.

```
UPDATE product
SET (quantity, version) = (4, 2)
WHERE id = 1 AND version = 1
```

- If the statement update count is zero, the version was incremented in the meanwhile, and the current transaction now operates on a stale record version.

The previous example can be adjusted to take advantage of this optimistic concurrency control mechanism. This time, the product is versioned, and both the web application and the batch processor data access logic are using the row versions to coordinate the *happens-before* update ordering.

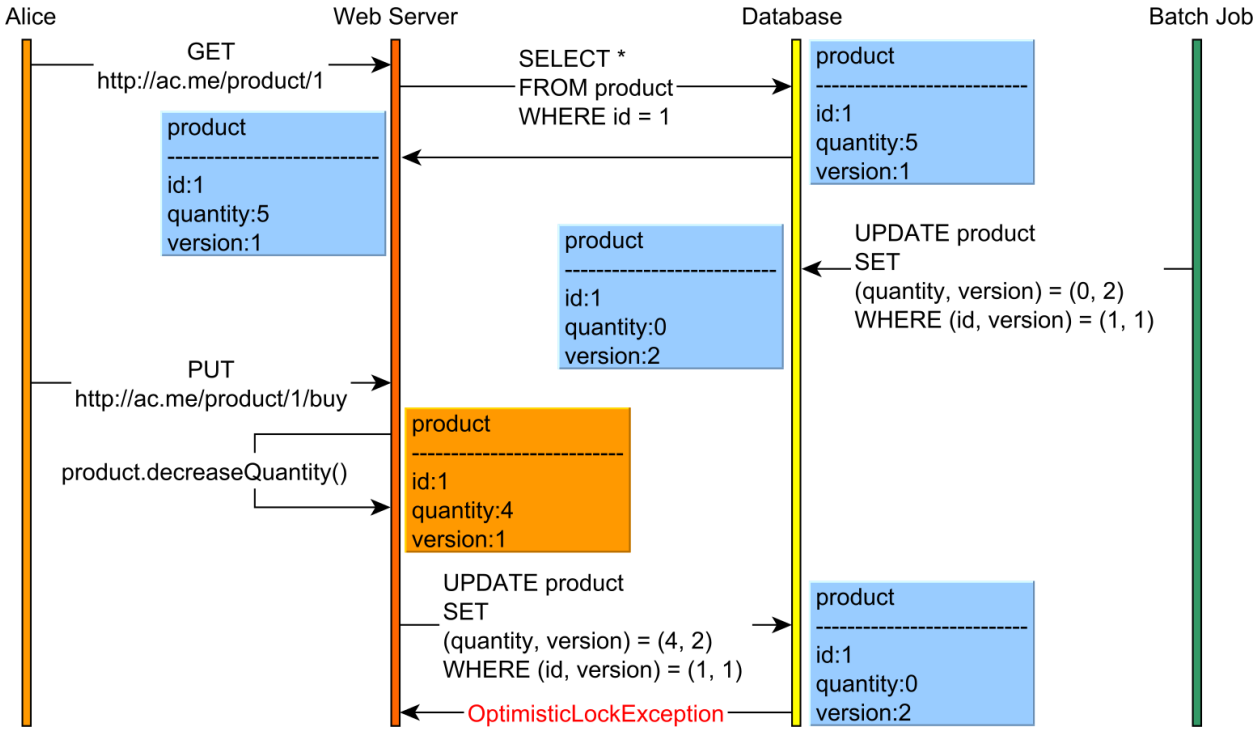

Figure 6.16: Stateful conversation preventing lost updates

Both *Alice* and the batch processor try to increment the product version optimistically. The batch processor can successfully update the product quantity since the SQL statement filtering criteria matches the actual database record version.

When *Alice* tries to update the product, the database returns a zero update count, and, this way, she is notified about the concurrent update that happened in the meanwhile.

The lost update can be prevented because the application can abort the current transaction when being notified of the stale record versions.

II JPA and Hibernate

7. Why JPA and Hibernate matter

Although JDBC does a very good job of exposing a common API that hides the database vendor-specific communication protocol, it suffers from the following shortcomings:

- The API is undoubtedly verbose, even for trivial tasks.
- Batching is not transparent from the data access layer perspective, requiring a specific API than its non-batched statement counterpart.
- Lack of built-in support for explicit locking and optimistic concurrency control.
- For local transactions, the data access is tangled with transaction management semantics.
- Fetching joined relations requires additional processing to transform the `ResultSet` into Domain Models or DTO (Data Transfer Object) graphs.

Although the primary goal of an ORM (Object-Relational Mapping) tool is to automatically translate object state transitions into SQL statements, this chapter aims to demonstrate that Hibernate can address all the aforementioned JDBC shortcomings.

Java Persistence history

The EJB 1.1 release offered a higher-level persistence abstraction through scalable enterprise components, known as Entity Beans. Although the design looked good on paper, in reality, the heavyweight RMI-based implementation proved to be disastrous from a performance perspective. Neither the EJB 2.0 support for *local interfaces* could revive the Entity Beans popularity, and, due to high-complexity and vendor-specific implementation details, most projects chose JDBC instead.

Hibernate was born out of all the frustration of using the Entity Bean developing model. As an open-source project, Hibernate managed to gain a lot of popularity, and so it soon became the *de facto* Java persistence framework.

In response to all the criticism associated with Entity Bean persistence, the Java Community Process advanced a lightweight POJO-based approach, and the JDO specification was born. Although JDO is data source agnostic, being capable of operating with both relation databases as well as NoSQL or even flat files, it never managed to hit mainstream popularity. For this reason, the Java Community Process decided that EJB3 would be based on a new specification, inspired by Hibernate and TopLink, and JPA (Java Persistence API) became the standard Java persistence technology.

The morale of this is that persistence is a very complex topic, and it demands a great deal of knowledge of both the database and the data access usage patterns.